

KRITISKT: MATS FÖRTYDLIGANDEN (2026-06-11)

Mats har varit EXTREMT tydlig med vad som INTE ska finnas:

TA BORT ALLT DETTA:

- “Massa jävla skit med olika rutor”
- Rutor med sammanfattningar (Ordrar, Ordervärde, Behov/Kapacitet, Beläggning)
- Ordet “ordrar” - **DET FINNS BARA “UPPGIFTER”**
- Ordet “kapacitet” - **DET ÄR INTE RELEVANT HÄR**
- Beläggningsprocent
- Teamkapacitet-tabeller
- Tyngdpunkt över tid-tabeller
- KPI-kort
- Komplexa dashboard-layouter

DETTA ÄR VAD SOM SKA FINNAS:

EN ENKEL LISTA MED:

1. Uppgiftsnamn
2. Uppgiftsnummer
3. Önskad leveranstid
4. Geografisk plats

OCH:

- Filter (distrikt, sök)
- Karta som visar var uppgifterna är
- Möjlighet att tilldela uppgifter till team/vecka

SLUT. INGET MER.

VAD MATS INTE VILL SE (IMPLEMENTERA ALDRIG DETTA)

Dessa saker har implementerats av misstag tidigare och ska **ALDRIG** finnas:

1. Rutor med sammanfattningar

- “Ordrar: 0”
- “Ordervärde: 0 kr”
- “Behov / Kapacitet: 0 h / 168 h”
- “Beläggning: 0%”

2. Tabeller med team-kapacitet

- Kolumner: Team, Ordrrar, Behov (h), Kapacitet (h), Beläggning

3. Tyngdpunkt över tid-tabell

- Vecka, Tyngdpunkt-ort, Uppgifter, Ordervärde, Produktionstid

4. Ordet “ordrar” - Använd **BARA** “uppgifter”

5. Allt som har med “kapacitet” eller “beläggning” att göra

OM DU IMPLEMENTERAR NÅGOT AV DETTA HAR DU MISSFÖRSTÅTT.

REPLIT INSTRUKTION: GROVPLANERING - ENKEL SPECIFIKATION

SPARA SOM: `/home/ubuntu/replit_grovplanering_FINAL_spec.md`

1. VAD ÄR GROVPLANERING?

Grovplanering är en **enkel lista** över ej planerade uppgifter.

Syfte:

- Visa alla uppgifter som inte är planerade ännu
- Filtrera och söka efter uppgifter
- Se var uppgifterna är geografiskt (på en karta)
- Tilldela uppgifter till team

DET ÄR ALLT. INGET MER.

DETTA ÄR ALLT. INGEN ANNAN KOMPLEXITET.

3. DATAMODELL (FÖRENKLAD)

3.1 Tabell: uppgifter

```
CREATE TABLE uppgifter (
  id                UUID PRIMARY KEY,
  tenant_id         UUID NOT NULL,

  -- Grundläggande (DETTA ÄR VAD SOM VISAS)
  uppgiftsnummer    VARCHAR(50) NOT NULL, -- "#1234"
  uppgiftsnamn      VARCHAR(255) NOT NULL, -- "Sophämtning Klara Sjö 600L"

  -- Tid
  senast_utforande  DATE,                  -- Önskad leveranstid

  -- Koppling till objekt (för geografisk plats)
  objekt_id         UUID REFERENCES objekt(id),

  -- Status
  status            VARCHAR(50) DEFAULT 'ej_planerad',
  -- 'ej_planerad', 'grovplanerad', 'utford'

  -- Tilldelning
  tilldelat_team_id UUID REFERENCES teams(id),
  tilldelad_vecka   VARCHAR(10), -- '2026-W24'

  -- Tidsstämplar
  skapad_vid        TIMESTAMP DEFAULT NOW(),
  uppdaterad_vid    TIMESTAMP DEFAULT NOW()
);
```

3.2 Tabell: objekt

```
CREATE TABLE objekt (
  id                UUID PRIMARY KEY,
  tenant_id         UUID NOT NULL,

  namn              VARCHAR(255) NOT NULL,

  -- Adress (DETTA ÄR VAD SOM VISAS)
  adress            VARCHAR(255),          -- "Klarabergsgatan 23, Stockholm"
  postnummer        VARCHAR(10),
  ort               VARCHAR(100),

  -- Geografisk position (för karta)
  latitude          DECIMAL(10, 8),
  longitude         DECIMAL(11, 8),

  -- Distrikt (för filtrering)
  distrikt          VARCHAR(100)
);
```

3.3 Tabell: teams

```
CREATE TABLE teams (
  id          UUID PRIMARY KEY,
  tenant_id   UUID NOT NULL,

  namn        VARCHAR(255) NOT NULL,

  aktiv       BOOLEAN DEFAULT true
);
```

TA BORT dessa fält (de ska INTE finnas):

- ✗ ordervarde
- ✗ produktionstid
- ✗ kapacitet
- ✗ belaggnings
- ✗ behov

4. API ENDPOINTS (FÖRENKLAD)

4.1 GET /api/grovplanering/uppgifter

Hämta lista med ej planerade uppgifter.

Query Parameters:

- `distrikt` (optional): Filtrera på distrikt
- `soktext` (optional): Sök efter uppgift

Response:

```
{
  "uppgifter": [
    {
      "id": "abc123",
      "uppgiftsnummer": "#1234",
      "uppgiftsnamn": "Sophämtning Klara Sjö 600L",
      "adress": "Klarabergsgatan 23, Stockholm",
      "senastUtforande": "2026-06-20",
      "latitude": 59.3293,
      "longitude": 18.0686
    },
    {
      "id": "def456",
      "uppgiftsnummer": "#1235",
      "uppgiftsnamn": "Tömning BioMax 1100L",
      "adress": "Drottninggatan 45, Stockholm",
      "senastUtforande": "2026-06-22",
      "latitude": 59.3326,
      "longitude": 18.0649
    }
  ]
}
```

INGET MER. Inga sammanfattningar, inga beräkningar.

4.2 POST /api/grovplanering/tilldela

Tilldela uppgifter till team och vecka.

Request Body:

```
{
  "uppgiftIds": ["abc123", "def456"],
  "teamId": "team-xyz",
  "vecka": "2026-W24"
}
```

Response:

```
{
  "success": true,
  "message": "2 uppgifter tilldelade"
}
```

4.3 GET /api/grovplanering/distrikt

Hämta lista med distrikt (för filter).

Response:

```
{
  "distrikt": ["Stockholm", "Södertälje", "Göteborg", "Malmö"]
}
```

4.4 GET /api/teams

Hämta lista med team (för tilldelning).

Response:

```
{
  "teams": [
    { "id": "team-1", "namn": "Team A" },
    { "id": "team-2", "namn": "Team B" }
  ]
}
```

INGET MER.

5. FRONTEND IMPLEMENTATION

5.1 Huvudkomponent

```
// app/grovplanering/page.tsx

'use client';

import { useState, useEffect } from 'react';

interface Uppgift {
  id: string;
  uppgiftsnummer: string;
  uppgiftsnamn: string;
  adress: string;
  senastUtforande: string;
  latitude: number;
  longitude: number;
}

export default function Grovplanering() {
  const [uppgifter, setUppgifter] = useState<Uppgift[]>([]);
  const [filterDistrikt, setFilterDistrikt] = useState<string>('');
  const [soktext, setSoktext] = useState<string>('');
  const [distrikt, setDistrikt] = useState<string[]>([]);
  const [teams, setTeams] = useState<any[]>([]);
  const [valdaUppgifter, setValdaUppgifter] = useState<Set<string>>(new Set());

  // Hämta uppgifter
  useEffect(() => {
    async function hamtaData() {
      const params = new URLSearchParams();
      if (filterDistrikt) params.append('distrikt', filterDistrikt);
      if (soktext) params.append('soktext', soktext);

      const res = await fetch(`/api/grovplanering/uppgifter?${params}`);
      const data = await res.json();
      setUppgifter(data.uppgifter);
    }
    hamtaData();
  }, [filterDistrikt, soktext]);

  // Hämta distrikt
  useEffect(() => {
    async function hamtaDistrikt() {
      const res = await fetch('/api/grovplanering/distrikt');
      const data = await res.json();
      setDistrikt(data.distrikt);
    }
    hamtaDistrikt();
  }, []);

  // Hämta teams
  useEffect(() => {
    async function hamtaTeams() {
      const res = await fetch('/api/teams');
      const data = await res.json();
      setTeams(data.teams);
    }
    hamtaTeams();
  }, []);

  // Tilldela uppgifter
  async function tilldelaUppgifter(teamId: string, vecka: string) {
    const res = await fetch('/api/grovplanering/tilldela', {
      method: 'POST',

```



```

headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({
  uppgiftIds: Array.from(valdaUppgifter),
  teamId,
  vecka
})
});

if (res.ok) {
  alert('Uppgifter tilldelade!');
  setValdaUppgifter(new Set());
  // Uppdatera listan
  const params = new URLSearchParams();
  if (filterDistrikt) params.append('distrikt', filterDistrikt);
  if (soktext) params.append('soktext', soktext);
  const nyRes = await fetch(`/api/grovplanering/uppgifter?${params}`);
  const nyData = await nyRes.json();
  setUppgifter(nyData.uppgifter);
}
}

// Toggle uppgift
function toggleUppgift(id: string) {
  const nya = new Set(valdaUppgifter);
  if (nya.has(id)) {
    nya.delete(id);
  } else {
    nya.add(id);
  }
  setValdaUppgifter(nya);
}

return (
  <div className="p-6">
    <h1 className="text-2xl font-bold mb-6"><img alt="calendar icon" data-bbox="521 531 541 546"/> Grovplanering</h1>

    {/ * FILTER */}
    <div className="mb-6 flex gap-4">
      <div>
        <label className="block text-sm font-medium mb-2">Filter:</label>
        <select
          value={filterDistrikt}
          onChange={(e) => setFilterDistrikt(e.target.value)}
          className="border rounded px-3 py-2"
        >
          <option value="">Alla distrikt</option>
          {distrikt.map((d) => (
            <option key={d} value={d}>
              {d}
            </option>
          ))}
        </select>
      </div>

      <div className="flex-1">
        <label className="block text-sm font-medium mb-2">Sök:</label>
        <input
          type="text"
          value={soktext}
          onChange={(e) => setSoktext(e.target.value)}
          placeholder="🔍 Sök uppgift..."
          className="border rounded px-3 py-2 w-full"
        />
      </div>
    </div>
  </div>

```

```

    </div>
  </div>

  { /* UPPGIFTSLISTA */ }
  <div className="mb-6">
    <h2 className="text-xl font-semibold mb-4">
      Ej planerade uppgifter ({uppgifter.length})
    </h2>

    <div className="space-y-3">
      {uppgifter.map((uppgift) => (
        <div
          key={uppgift.id}
          className="border rounded p-4 hover:bg-gray-50"
        >
          <div className="flex items-start gap-3">
            <input
              type="checkbox"
              checked={valdaUppgifter.has(uppgift.id)}
              onChange={() => toggleUppgift(uppgift.id)}
              className="mt-1"
            />

            <div className="flex-1">
              <div className="font-semibold">
                {uppgift.uppgiftsnummer} - {uppgift.uppgiftsnamn}
              </div>
              <div className="text-sm text-gray-600 mt-1">
                📍 {uppgift.adress}
              </div>
              <div className="text-sm text-gray-600">
                📅 Senast: {uppgift.senastUtforande}
              </div>
            </div>

            <button className="px-3 py-1 bg-blue-500 text-white rounded text-sm">
              Tilldela team 📌
            </button>
          </div>
        </div>
      ))}
    </div>
  </div>

  { /* KARTA */ }
  <div className="border rounded p-4">
    <h2 className="text-xl font-semibold mb-4">🗺️ Karta</h2>
    <div className="bg-gray-100 h-96 flex items-center justify-center text-gray-500">
      [Karta som visar pins för alla uppgifter kommer här]
    <br />
    Använd bibliotek som react-leaflet eller Mapbox
  </div>
</div>
);
}

```

5.2 Viktiga punkter för implementationen

GÖR:

- ☒ EN enkel lista med uppgifter

- ☒ Visa uppgiftsnummer, uppgiftsnamn, adress, leveranstid
- ☒ Filter för distrikt
- ☒ Sökfält
- ☒ Karta med pins
- ☒ Knapp för att tilldela uppgifter

GÖR INTE:

- ☒ Rutor med sammanfattningar
- ☒ Använd ordet "ordrar"
- ☒ Visa "kapacitet"
- ☒ Visa "beläggning"
- ☒ Komplexa tabeller
- ☒ KPI-kort
- ☒ Dashboard-widgets

6. KARTA IMPLEMENTATION

Använd **react-leaflet** för att visa uppgifter på en karta.

```
// components/GrovplaneringKarta.tsx

'use client';

import { MapContainer, TileLayer, Marker, Popup } from 'react-leaflet';
import 'leaflet/dist/leaflet.css';

interface Uppgift {
  id: string;
  uppgiftsnummer: string;
  uppgiftsnamn: string;
  adress: string;
  latitude: number;
  longitude: number;
}

interface Props {
  uppgifter: Uppgift[];
}

export default function GrovplaneringKarta({ uppgifter }: Props) {
  // Standardposition (Stockholm)
  const center: [number, number] = [59.3293, 18.0686];

  return (
    <MapContainer
      center={center}
      zoom={12}
      style={{ height: '400px', width: '100%' }}
    >
      <TileLayer
        url="https://upload.wikimedia.org/wikipedia/commons/thumb/f/f2/Tiled_web_map_numbering.png/330px-Tiled_web_map_numbering.png?utm_source=commons.wikimedia.org&utm_campaign=imageinfo&utm_content=thumbnail"
        attribution='&copy; <a href="https://www.openstreetmap.org/copy-right">OpenStreetMap</a>'
      />

      {uppgifter.map((uppgift) => {
        if (!uppgift.latitude || !uppgift.longitude) return null;

        return (
          <Marker
            key={uppgift.id}
            position={[uppgift.latitude, uppgift.longitude]}
          >
            <Popup>
              <div>
                <strong>{uppgift.uppgiftsnummer}</strong>
                <br />
                {uppgift.uppgiftsnamn}
                <br />
                📍 {uppgift.adress}
              </div>
            </Popup>
          </Marker>
        );
      })}
    </MapContainer>
  );
}
```

Installation:

```
npm install react-leaflet leaflet  
npm install -D @types/leaflet
```

7. BACKEND API IMPLEMENTATION

7.1 GET /api/grovplanering/uppgifter

```
// app/api/grovplanering/uppgifter/route.ts

import { NextRequest, NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';
import { getServerSession } from 'next-auth';
import { authOptions } from '@lib/auth';

export async function GET(req: NextRequest) {
  // Autentisering
  const session = await getServerSession(authOptions);
  if (!session?.user?.tenantId) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const tenantId = session.user.tenantId;

  // Query parameters
  const { searchParams } = new URL(req.url);
  const distrikt = searchParams.get('distrikt') || '';
  const soktext = searchParams.get('soktext') || '';

  // Bygg query
  const where: any = {
    tenantId,
    status: 'ej_planerad'
  };

  // Filter på distrikt
  if (distrikt) {
    where.objekt = {
      distrikt
    };
  }

  // Sök
  if (soktext) {
    where.OR = [
      { uppgiftsnummer: { contains: soktext, mode: 'insensitive' } },
      { uppgiftsnamn: { contains: soktext, mode: 'insensitive' } },
      {
        objekt: {
          adress: { contains: soktext, mode: 'insensitive' }
        }
      }
    ];
  }

  // Hämta uppgifter
  const uppgifter = await prisma.uppgift.findMany({
    where,
    include: {
      objekt: {
        select: {
          adress: true,
          latitude: true,
          longitude: true
        }
      }
    },
    orderBy: {
      skapadVid: 'desc'
    }
  });
}
```

```
});

// Formatera response
const formatted = uppgifter.map((u) => ({
  id: u.id,
  uppgiftsnummer: u.uppgiftsnummer,
  uppgiftsnamn: u.uppgiftsnamn,
  adress: u.objekt?.adress || '',
  senastUtforande: u.senastUtforande?.toISOString().split('T')[0] || '',
  latitude: u.objekt?.latitude || 0,
  longitude: u.objekt?.longitude || 0
}));

return NextResponse.json({ uppgifter: formatted });
}
```


7.2 POST /api/grovplanering/tilldelade

```
// app/api/grovplanering/tilldelade/route.ts

import { NextRequest, NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';
import { getSession } from 'next-auth';
import { authOptions } from '@lib/auth';

export async function POST(req: NextRequest) {
  // Autentisering
  const session = await getSession(authOptions);
  if (!session?.user?.tenantId) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const tenantId = session.user.tenantId;

  // Body
  const body = await req.json();
  const { uppgiftIds, teamId, vecka } = body;

  // Validering
  if (!uppgiftIds || !Array.isArray(uppgiftIds) || uppgiftIds.length === 0) {
    return NextResponse.json({ error: 'uppgiftIds saknas' }, { status: 400 });
  }

  if (!teamId) {
    return NextResponse.json({ error: 'teamId saknas' }, { status: 400 });
  }

  // Uppdatera uppgifter
  await prisma.uppgift.updateMany({
    where: {
      id: { in: uppgiftIds },
      tenantId,
      status: 'ej_planerad'
    },
    data: {
      tilldelatTeamId: teamId,
      tilldeladVecka: vecka,
      status: 'grovplanerad',
      uppdateradVid: new Date()
    }
  });

  return NextResponse.json({
    success: true,
    message: `${uppgiftIds.length} uppgifter tilldelade`
  });
}
```

7.3 GET /api/grovplanering/distrikt

```
// app/api/grovplanering/distrikt/route.ts

import { NextRequest, NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';
import { getServerSession } from 'next-auth';
import { authOptions } from '@lib/auth';

export async function GET(req: NextRequest) {
  const session = await getServerSession(authOptions);
  if (!session?.user?.tenantId) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const tenantId = session.user.tenantId;

  // Hämta unika distrikt
  const objekt = await prisma.objekt.findMany({
    where: {
      tenantId,
      distrikt: { not: null }
    },
    select: {
      distrikt: true
    },
    distinct: ['distrikt']
  });

  const distrikt = objekt
    .map((o) => o.distrikt)
    .filter(Boolean)
    .sort();

  return NextResponse.json({ distrikt });
}
```

7.4 GET /api/teams

```
// app/api/teams/route.ts

import { NextRequest, NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';
import { getServerSession } from 'next-auth';
import { authOptions } from '@lib/auth';

export async function GET(req: NextRequest) {
  const session = await getServerSession(authOptions);
  if (!session?.user?.tenantId) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const tenantId = session.user.tenantId;

  const teams = await prisma.team.findMany({
    where: {
      tenantId,
      aktiv: true
    },
    select: {
      id: true,
      namn: true
    },
    orderBy: {
      namn: 'asc'
    }
  });

  return NextResponse.json({ teams });
}
```

8. PRISMA SCHEMA (FÖRENKLAD)

```
// prisma/schema.prisma

model Uppgift {
  id          String    @id @default(cuid())
  tenantId    String

  // Grundläggande
  uppgiftsnummer String
  uppgiftsnamn   String

  // Tid
  senastUtforande DateTime?

  // Kopplingar
  objektId    String?
  objekt      Objekt? @relation(fields: [objektId], references: [id])

  // Status
  status      String    @default("ej_planerad")

  // Tilldelning
  tilldelatTeamId String?
  tilldeladVecka  String?
  team           Team?   @relation(fields: [tilldelatTeamId], references: [id])

  // Tidsstämplar
  skapadVid     DateTime @default(now())
  uppdateradVid DateTime @updatedAt

  @@index([tenantId, status])
  @@map("uppgifter")
}

model Objekt {
  id          String    @id @default(cuid())
  tenantId    String

  namn        String

  // Adress
  adress      String?
  postnummer  String?
  ort         String?

  // Geografisk position
  latitude    Float?
  longitude   Float?

  // Distrikt
  distrikt    String?

  // Relations
  uppgifter   Uppgift[]

  @@index([tenantId, distrikt])
  @@map("objekt")
}

model Team {
  id          String    @id @default(cuid())
  tenantId    String
```

```

namn      String

aktiv      Boolean  @default(true)

// Relations
uppgifter Uppgift[]

@@index([tenantId, aktiv])
@@map("teams")

```

9. IMPLEMENTATIONSCHECKLISTA

✓ Före du börjar implementera - LÄSKONTROLL:

- ☐ Har jag läst HELA detta dokument?
- ☐ Har jag förstått att det ska vara EN ENKEL LISTA?
- ☐ Har jag förstått att det INTE ska finnas:
- ☐ Rutor med sammanfattningar
- ☐ Ordet "ordrar" (använd BARA "uppgifter")
- ☐ Ordet "kapacitet"
- ☐ Beläggningsprocent
- ☐ KPI-kort

✓ Backend:

- ☐ Databastabeller skapade (uppgifter, objekt, teams)
- ☐ Prisma schema konfigurerat
- ☐ API endpoint: GET /api/grovplanering/uppgifter
- ☐ API endpoint: POST /api/grovplanering/tilldela
- ☐ API endpoint: GET /api/grovplanering/distrikt
- ☐ API endpoint: GET /api/teams

✓ Frontend:

- ☐ Huvudsida: app/grovplanering/page.tsx
- ☐ Filter (distrikt dropdown + sökfält)
- ☐ Uppgiftslista (enkel lista med kort)
- ☐ Checkbox för att välja uppgifter
- ☐ Knapp för att tilldela uppgifter
- ☐ Karta med pins (react-leaflet)

✓ Efter implementation - KVALITETSKONTROLL:

- ☐ Har jag tagit bort ALLT onödigt?
- ☐ Är det en ENKEL lista?
- ☐ Använder jag ordet "uppgifter" (INTE "ordrar")?
- ☐ Finns det INGA rutor med sammanfattningar?
- ☐ Finns det INGEN kapacitet?
- ☐ Finns det INGEN beläggning?
- ☐ Visar listan ENDAST: uppgiftsnamn, nummer, leveranstid, plats?

- [] Finns det EN karta som visar var uppgifterna är?
- [] Kan jag tilldela uppgifter till team?

KRITISK KONTROLL:

OM DU SVARAR "JA" PÅ NÅGON AV DESSA FRÅGOR HAR DU IMPLEMENTERAT FEL:

- [] Finns det rutor som visar "Ordrrar: 0"?
- [] Finns det rutor som visar "Ordervärde"?
- [] Finns det rutor som visar "Kapacitet"?
- [] Finns det rutor som visar "Beläggning"?
- [] Finns det tabeller med team-kapacitet?
- [] Används ordet "ordrar" någonstans?

OM DU SVARADE "JA" PÅ NÅGON FRÅGA OVAN: RADERA OCH BÖRJA OM.

10. SAMMANFATTNING

Grovplanering är:

- EN enkel lista med uppgifter
- Ett filter (distrikt + sök)
- En karta som visar var uppgifterna är
- Möjlighet att tilldela uppgifter till team

Grovplanering är INTE:

- Komplexa rutor med sammanfattningar
- Kapacitet eller beläggning
- Ordervärde eller produktionstid
- Dashboard med KPI:er

GÖR DET ENKELT. INGET MER.

11. EXEMPEL PÅ FEL IMPLEMENTATION (GÖR INTE SÅ HÄR)

✗ EXEMPEL 1: Komplexa rutor (FEL!)

```
// GÖR INTE SÅ HÄR!  
<div className="grid grid-cols-4 gap-4">  
  <div className="border p-4">  
    <h3>Ordrrar</h3>  
    <p className="text-2xl">0</p>  
  </div>  
  <div className="border p-4">  
    <h3>Ordervärde</h3>  
    <p className="text-2xl">0 kr</p>  
  </div>  
  <div className="border p-4">  
    <h3>Kapacitet</h3>  
    <p className="text-2xl">168 h</p>  
  </div>  
  <div className="border p-4">  
    <h3>Beläggning</h3>  
    <p className="text-2xl">0%</p>  
  </div>  
</div>
```

DETTA ÄR FEL! IMPLEMENTERA INTE DETTA!

✓ EXEMPEL 2: Korrekt implementation

```
// GÖR SÅ HÄR ISTÄLLET!
<div>
  <h1> Grovplanering</h1>

  <div className="mb-4">
    <select className="border p-2">
      <option>Alla distrikt</option>
      <option>Stockholm</option>
    </select>
    <input
      type="text"
      placeholder="🔍 Sök uppgift..."
      className="border p-2 ml-2"
    />
  </div>

  <h2>Ej planerade uppgifter (45)</h2>

  <div className="space-y-2">
    {uppgifter.map(u => (
      <div key={u.id} className="border p-3">
        <input type="checkbox" />
        <strong>{u.uppgiftsnummer} - {u.uppgiftsnamn}</strong>
        <div> {u.adress}</div>
        <div> Senast: {u.senastUtforande}</div>
      </div>
    ))}
  </div>

  <div className="mt-6">
    <h2> Karta</h2>
    <Map uppgifter={uppgifter} />
  </div>
</div>
```

DETTA ÄR RÄTT! ENKELT OCH TYDLIGT!

SLUTORD TILL REPLIT

Mats har varit EXTREMT tydlig. Han vill ha det ENKELT.

EN ENKEL LISTA. EN KARTA. FILTER. TILLDELA UPPGIFTER.

INGET MER.

OM DU IMPLEMENTERAR NÅGOT ANNAT HAR DU MISSFÖRSTÅTT.

LÄS DETTA DOKUMENT IGEN OM DU ÄR OSÄKER.

END OF SPECIFICATION